
LAPORAN PRAKTIKUM INTERNET OF THINGS II



Disusun Oleh:

Nama: M Riki Pratama

Kelas: Teknologi Rekayasa Internet (3C)

NPM: 25782072

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

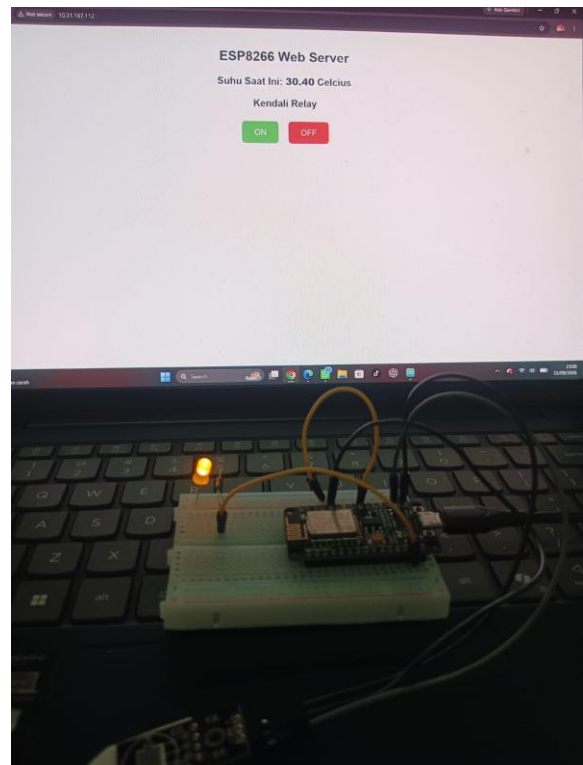
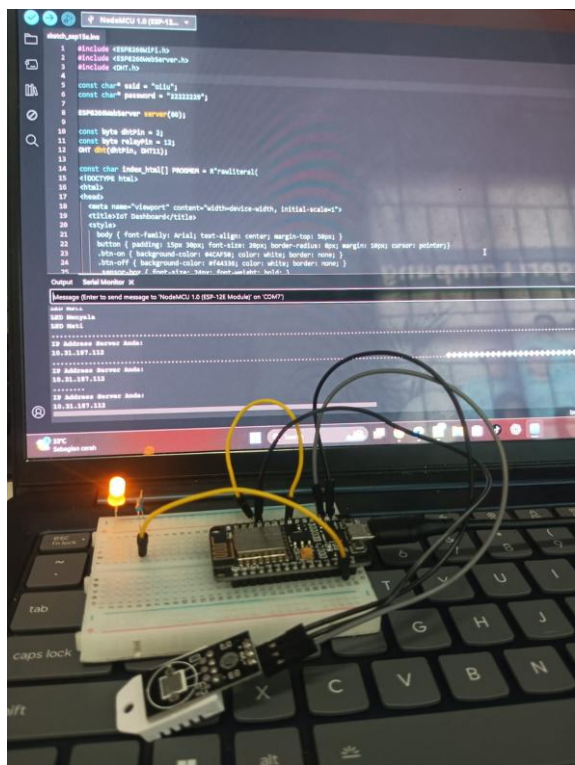
POLITEKNIK NEGERI LAMPUNG 2026

Modul 3: Jaringan Nirkabel dan Komunikasi HTTP

3.5 Praktikum

3.5.1 Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)

Skenario: Anda merakit NodeMCU sebagai server pintar. Modul ini membaca sensor DHT dan menampilkan suhunya di sebuah halaman web (UI), sekaligus menyediakan tombol di web tersebut untuk menghidupkan/mematikan lampu LED.



Langkah Kerja Rangkaian:

1. Hubungkan pin **DATA / OUT / S** DHT ke pin **D4 (GPIO 2)**. (Pastikan VCC DHT ke 3V3, GND ke GND).
2. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**. Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND** NodeMCU. (Catatan: LED ini dikendalikan oleh pin D6 / GPIO 12 yang pada kode program didefinisikan sebagai relayPin).

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*). Tuliskan kode program di bawah. **Jangan lupa**

mengganti variabel ssid dan password!

2. *Catatan teknis:* Kita menggunakan teknik **Raw String Literal C++** (`R"rawliteral(...)rawliteral"`) untuk menulis kode antarmuka HTML multibaris, sehingga kita tidak repot mengetikkan tanda kutip `""` di setiap baris HTML.

```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "oiuu";
6 const char* password = "22222229";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT22);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18 <meta name="viewport" content="width=device-width, initial-scale=1">
19 <title>IoT Dashboard</title>
20 <style>
21   body { font-family: Arial; text-align: center; margin-top: 50px; }
22   button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer; }
23   .btn-on { background-color: #4CAF50; color: white; border: none; }
24   .btn-off { background-color: #f44336; color: white; border: none; }
25   .sensor-box { font-size: 24px; font-weight: bold; }
26 </style>
27 </head>
28 <body>
29 <h1>ESP8266 Web Server</h1>
30 <div class="sensor-box">
31   <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
32 </div>
33 <h2>Kendali Relay</h2>
34 <a href="/relay/on"><button class="btn-on">ON</button></a>
35 <a href="/relay/off"><button class="btn-off">OFF</button></a>
36 </body>
37 </html>
38 )rawliteral";
```

```
40 void handleRoot() {
41   String html = index_html; // Salin kerangka HTML ke variabel dinamis
42   float t = dht.readTemperature();
43
44   // Mengganti teks placeholder %TEMPERATURE% dengan suhu nyata
45   if (isnan(t)) {
46     html.replace("%TEMPERATURE%", "--");
47   } else {
48     html.replace("%TEMPERATURE%", String(t));
49   }
50
51   server.send(200, "text/html", html);
52 }
53
54 void handleRelayOn() {
55   digitalWrite(relayPin, HIGH);
56   server.sendHeader("Location", "/");
57   server.send(303);
58 }
59
60 void handleRelayOff() {
61   digitalWrite(relayPin, LOW);
62   server.sendHeader("Location", "/");
63   server.send(303);
64 }
65
66 void setup() {
67   Serial.begin(115200);
68   pinMode(relayPin, OUTPUT);
69   digitalWrite(relayPin, LOW);
70   dht.begin();
71
72   WiFi.mode(WIFI_STA);
73   WiFi.begin(ssid, password);
74   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
75   Serial.println("\nIP Address Server Anda: ");
76   Serial.println(WiFi.localIP());
77
78   server.on("/", handleRoot);
79   server.on("/relay/on", handleRelayOn);
80   server.on("/relay/off", handleRelayOff);
81 }
```

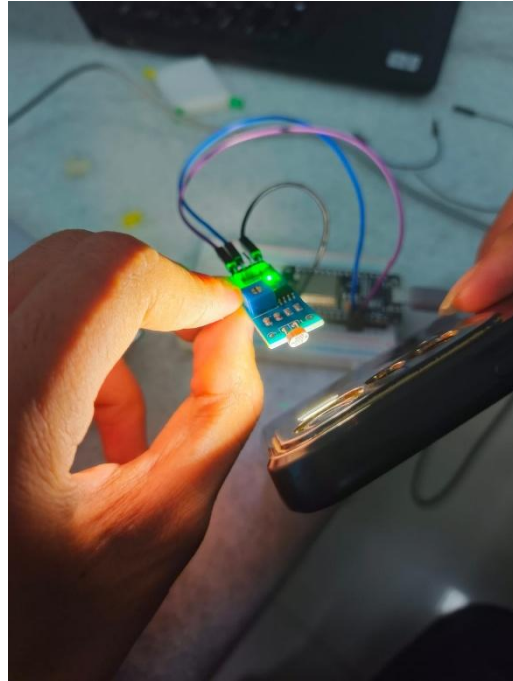
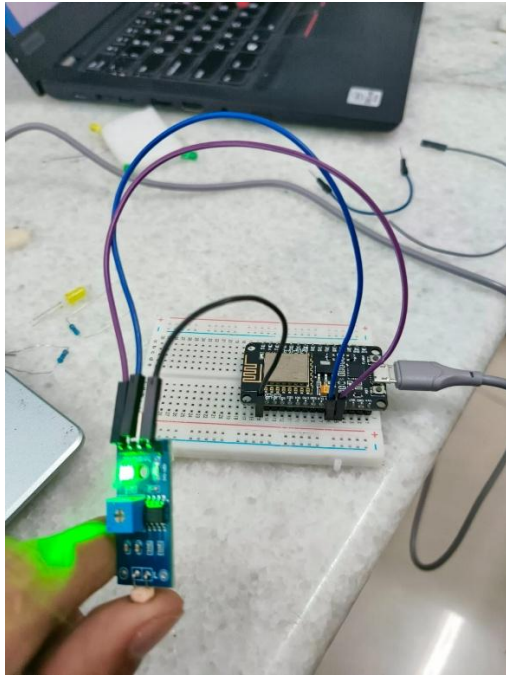
3. **Upload** kode tersebut. Buka **Serial Monitor**, dan catat alamat IP yang didapat (10.31.187.112).
4. Hubungkan Laptop/HP Anda ke Wi-Fi yang sama, buka *browser* (Chrome/Safari), lalu ketikkan alamat IP tersebut. Klik tombol ON dan OFF di layar untuk melihat LED menyala dan padam.

Penjelasan Singkat Kode:

- `ESP8266WebServer server(80);`: Menginisialisasi *library Server* agar terus mendengarkan panggilan di jalur lalu lintas standar web (*Port 80*).
- `html.replace("%TEMPERATURE%", String(t));`: Fitur manipulasi teks (*String*). Perintah ini secara cerdas akan menyapu dokumen HTML, mencari teks *placeholder* `%TEMPERATURE%`, dan menggantinya dengan variabel angka suhu yang didapat dari sensor aktual sebelum dikirim ke layar pengunjung. Inilah esensi dari halaman *web* dinamis.
- `server.on("/relay/on", handleRelayOn);`: Fungsi *Routing*. Jika ada klien mengunjungi tautan URL berakhiran `/relay/on`, server akan menjalankan aksi mengubah *state* LED (menyalakan/mematikan output), lalu me-*redirect* layar pengunjung kembali ke halaman depan (*Location: /*) via kode 303.
- `server.handleClient();`: Berada di dalam `loop()` untuk terus-menerus mengecek apakah ada permintaan pengunjung yang datang setiap milidetiknya.

3.5.2 Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan)

Skenario: Anda sekarang memprogram NodeMCU kedua (atau meminta teman kelompok di sebelah Anda) untuk menjadi *Client*. Perangkat Klien ini dipasang LDR. Jika LDR tertutup kegelapan, ia akan mengirim sinyal HTTP secara jarak jauh (via jaringan Wi-Fi) untuk menyalakan LED di *Server* milik teman Anda (Praktikum 1).



Langkah Kerja Rangkaian:

1. Rangkai sensor LDR dan Resistor 10k Ohm sebagai pembagi tegangan (seperti di Modul 2) lalu hubungkan jalur sinyalnya ke pin A0.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*).
2. Tuliskan kode program di bawah. Ganti variabel `serverName` dengan IP Address NodeMCU Praktikum 1 milik teman sebelah Anda. (Pastikan Anda menggunakan Wi-Fi yang sama persis).

```

1  #include <ESP8266WiFi.h>
2  #include <ESP8266HTTPClient.h>
3  #include <WiFiClient.h>
4
5  const char* ssid = "oiiu";
6  const char* password = "22222229";
7
8  const char* serverName = "http://10.31.187.112/relay/on";
9  const byte ldrPin = A0;
10
11 void setup() {
12     Serial.begin(115200);
13     WiFi.mode(WIFI_STA);
14     WiFi.begin(ssid, password);
15     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16     Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
18
19 void loop() {
20     int ldrValue = analogRead(ldrPin);
21
22     if ((WiFi.status() == WL_CONNECTED) && (ldrValue > 300)) {
23         WiFiClient client;
24         HTTPClient http;
25
26         http.begin(client, serverName);
27         int httpStatusCode = http.GET();
28
29         Serial.print("HTTP Response code: ");
30         Serial.println(httpStatusCode);
31
32         http.end();
33         delay(10000);
34     }
35     delay(2000);
36 }

```

3. Upload kode tersebut, dan buka Serial Monitor. Saat LDR Anda di *Client* ditutup tangan hingga gelap, perhatikan LED di NodeMCU *Server* milik teman Anda pasti akan menyala!

Penjelasan Singkat Kode:

- **WiFiClient client;** dan **HTTPClient http;**: Kelas pemanggil untuk memulai eksekusi permintaan *client* jarak jauh (mirip fungsi aplikasi *browser* di laptop).
- **http.begin(client, serverName);**: Menargetkan sasaran URL *server*.
- **int httpStatusCode = http.GET();**: Mengeksekusi ketukan pintu (*Request HTTP GET*). Jika *Server* merespons sukses, nilai variabel ini akan berisi angka standar jaringan seperti 200 (OK) atau 303 (Redirect). Jika URL rusak atau *Server offline*, variabel akan mereturn nilai negatif.
- **delay(10000);**: *Cooldown* panjang 10 detik ini ditambahkan agar *Client* tidak

memborbardir (*flood/DDoS*) Server dengan permintaan on secara terus-menerus setiap detik ketika kondisinya sedang gelap.

3.6 Latihan

Kerjakan soal berikut secara mandiri untuk menguji penalaran dan modifikasi jaringan.

- **Latihan 1 (Pemahaman Jaringan):** Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

Jawaban: Menggunakan mode Wi-Fi Station (STA) karena NodeMCU ESP8266 berperan sebagai client yang terhubung ke jaringan Wi-Fi yang sudah tersedia, kemudian menjalankan Web Server yang dapat diakses oleh perangkat lain dalam jaringan tersebut. Dengan mode STA, NodeMCU mendapatkan alamat IP dari router atau hotspot sehingga browser pada perangkat lain dapat mengakses Web Server melalui alamat IP tersebut.

Sedangkan pada mode Access Point (AP), NodeMCU membuat dan menyediakan jaringan Wi-Fi sendiri. Mode ini tidak digunakan pada Praktikum karena praktikumnya menggunakan jaringan Wi-Fi yang sudah tersedia untuk menghubungkan NodeMCU dengan perangkat client.

- **Latihan 2 (Modifikasi Antarmuka & Telemetry):** Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:
 - i. **Menampilkan Kelembapan:** Tambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">` (contoh: `<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>`), lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman *web*.

```

40 void handleRoot() {
41     String html = index_html;
42
43     float t = dht.readTemperature();
44     float h = dht.readHumidity();
45
46     // Menampilkan suhu
47     if (isnan(t)) {
48         html.replace("%TEMPERATURE%", "--");
49     } else {
50         html.replace("%TEMPERATURE%", String(t));
51     }
52
53     // Menampilkan kelembapan
54     if (isnan(h)) {
55         html.replace("%HUMIDITY%", "--");
56     } else {
57         html.replace("%HUMIDITY%", String(h));
58     }
59
60     // Membuat tombol dinamis berdasarkan kondisi relay/LED
61     if (digitalRead(relayPin) == LOW) {
62         html.replace(
63             "%RELAY_BUTTON%",
64             "<a href=\"/relay/on\"><button class=\"btn-on\">NYALAKAN (ON)</button></a>"
65         );
66     } else {
67         html.replace(
68             "%RELAY_BUTTON%",
69             "<a href=\"/relay/off\"><button class=\"btn-off\">MATIKAN (OFF)</button></a>"
70         );
71     }
72
73     server.send(200, "text/html", html);
74 }

```

```

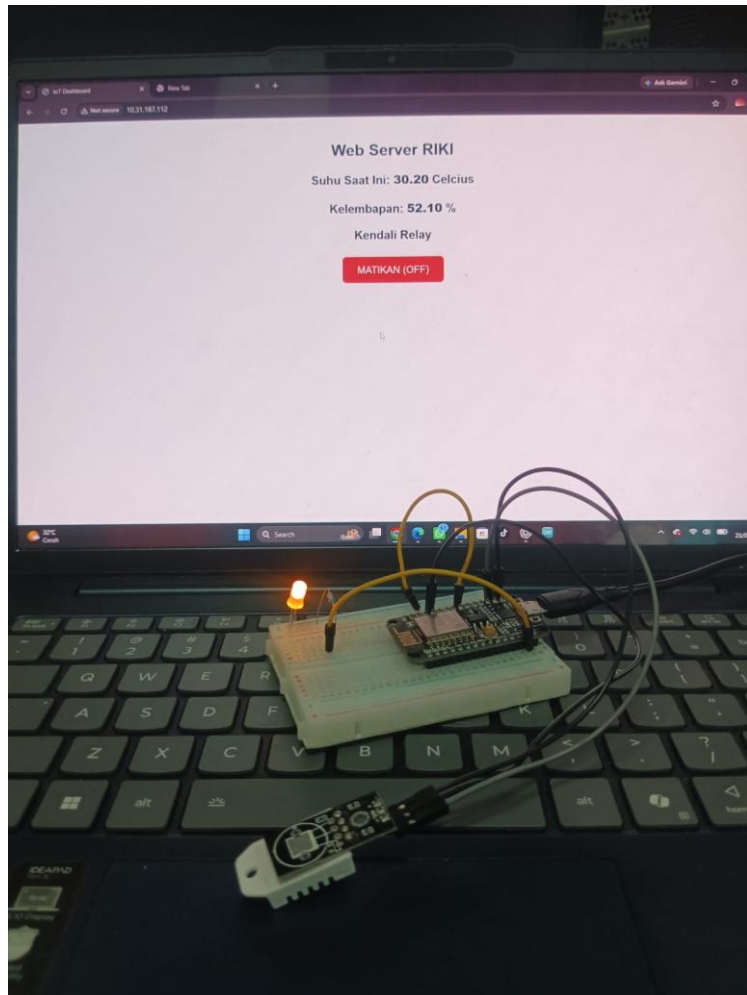
30 <div class="sensor-box">
31     <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
32     <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
33 </div>
34 <h2>Kendali Relay</h2>
35 %RELAY_BUTTON%
36 </body>
37 </html>
38 )rawliteral";

```

ii. **Tombol Dinamis Tunggal (*Toggle Button*):** Ubah antarmuka agar hanya menggunakan 1 buah tombol tunggal yang statusnya adaptif mengikuti kondisi LED saat ini:

- Jika LED sedang OFF, tombol menampilkan tulisan "NYALAKAN (ON)" (misal: warna hijau/biru) dan mengarahkan aksi ke /relay/on.
- Jika LED sedang ON, tombol otomatis berubah menjadi tulisan "MATIKAN (OFF)" (warna merah) dan mengarahkan aksi ke /relay/off.
(Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti %RELAY_BUTTON%, lalu di fungsi

handleRoot(), periksa status pin relay/LED menggunakan *digitalRead(relayPin)* sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai). Tuliskan potongan baris kode HTML dan fungsi *handleRoot()* yang Anda ubah!



- **Latihan 3 (Analisis Kode Client):** Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

```
Output: Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

.....
Client Terhubung ke Wi-Fi!
HTTP Response code: 303
HTTP Response code: -1
HTTP Response code: -1
```

- **Latihan 4 (Pengembangan Algoritma):** Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) HANYA JIKA sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

```
Output: Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Suhu: 32.60 C
Suhu: 32.60 C
Suhu: 32.60 C
Suhu: 32.60 C
Suhu: 32.70 C
Suhu: 32.60 C
Suhu: 32.70 C
Suhu: 32.70 C
Suhu: 32.70 C
Suhu: 32.30 C
Suhu > 30 C
Mengirim HTTP GET...
HTTP Response code: 303
Suhu: 32.20 C
Suhu > 30 C
Mengirim HTTP GET...
HTTP Response code: 303
```

```

1  #include <ESP8266WiFi.h>
2  #include <ESP8266HTTPClient.h>
3  #include <WiFiClient.h>
4  #include <DHT.h>
5
6  const char* ssid = "oiiu";
7  const char* password = "22222229";
8
9  const char* serverName = "http://10.31.187.112/relay/on";
10
11 const byte dhtPin = 2; // D4 / GPIO2
12 DHT dht(dhtPin, DHT22);
13
14 void setup() {
15     Serial.begin(115200);
16
17     dht.begin();
18
19     WiFi.mode(WIFI_STA);
20     WiFi.begin(ssid, password);
21
22     Serial.print("Menghubungkan ke Wi-Fi");
23
24     while (WiFi.status() != WL_CONNECTED) {
25         delay(500);
26         Serial.print(".");
27     }
28
29     Serial.println();
30     Serial.println("Client Terhubung ke Wi-Fi!");
31     Serial.print("IP Client: ");
32     Serial.println(WiFi.localIP());
33 }
34
35 void loop() {
36
37     // Membaca suhu dari DHT22
38     float temperature = dht.readTemperature();
39

```

```

37     // Membaca suhu dari DHT22
38     float temperature = dht.readTemperature();
39
40     // Mengecek apakah sensor berhasil dibaca
41     if (isnan(temperature)) {
42         Serial.println("Gagal membaca suhu DHT22!");
43         delay(2000);
44         return;
45     }
46
47     Serial.print("Suhu: ");
48     Serial.print(temperature);
49     Serial.println(" C");
50
51     // Jika suhu lebih dari 35 C, kirim HTTP GET
52     if (WiFi.status() == WL_CONNECTED && temperature > 30) {
53
54         WiFiClient client;
55         HTTPClient http;
56
57         Serial.println("Suhu > 30 C");
58         Serial.println("Mengirim HTTP GET...");
59
60         http.begin(client, serverName);
61
62         int httpResponseCode = http.GET();
63
64         Serial.print("HTTP Response code: ");
65         Serial.println(httpResponseCode);
66
67         http.end();
68
69         delay(10000);
70     }
71     else {
72         Serial.println("Suhu <= 30 C, tidak mengirim GET.");
73     }
74
75     delay(2000);
76 }

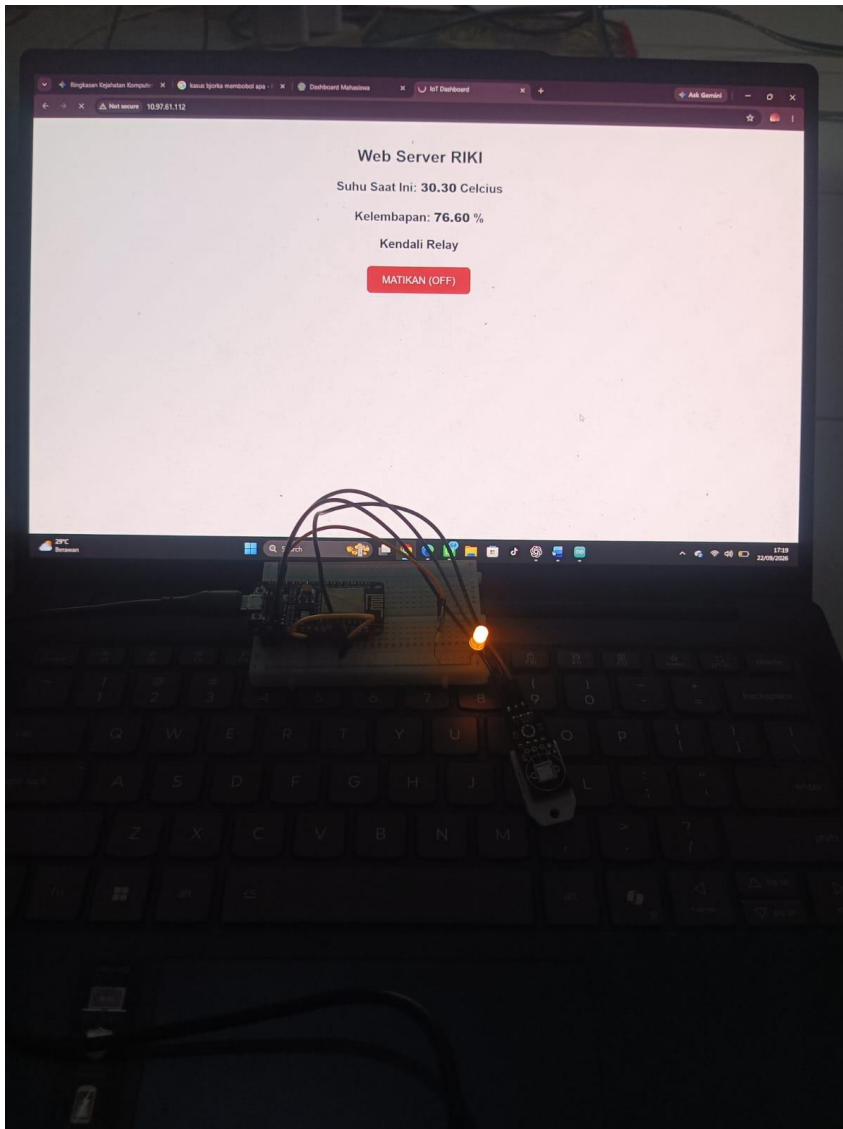
```

3.7 Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur

Antarmuka (*User Interface*) IoT tidak boleh statis. Saat ini, *Web Server* kita tidak mampu memperbarui informasi suhunya secara otomatis ke layar pengunjung tanpa diperintah. Kita harus menyelesaikannya.

Skenario Proyek (*Web Dashboard*): Anda akan bertindak sebagai *Full-Stack IoT Developer* untuk memutakhirkan kapabilitas *Web Server* pada Latihan 2, sembari mengevaluasi limitasi arsitektur tradisional HTTP.

- Tujuan: Mampu memanipulasi struktur kerangka HTML mentah yang tersimpan di memori NodeMCU untuk mencapai interaktivitas tampilan otomatis (*Auto-Refresh*).
- Tingkat Kesulitan: Menengah
- Instruksi:
 - i. Lakukan modifikasi kode hanya pada sisi NodeMCU Server (Latihan 2).
 - ii. Modifikasi variabel Raw Literal `index_html`. Sisipkan perintah *Meta Refresh HTML* (sebuah Tag standar web kuno namun ampuh) persis di dalam cakupan tag `<head>` untuk memaksa *browser* memuat ulang halaman secara sekuensial setiap 5 detik.
 - iii. *Hint (Petunjuk)*: Sintaks standar yang harus disisipkan adalah `<meta http-equiv="refresh" content="5">`.
 - iv. Setelah selesai, buka *browser*, dan amati layarnya. Biarkan selama 20 detik tanpa menyentuh *mouse/keyboard*.
- Instruksi Analisis Arsitektur:
 - i. Setelah Auto-Refresh berjalan, cobalah tekan tombol *ON / OFF* (kendali LED) dari *browser*.
 - ii. Apa kelemahan visual/interaksi (*flickering*) yang Anda alami saat menggunakan metode *Meta Refresh* HTTP konvensional seperti ini untuk sistem yang mengklaim diri sebagai *Real-Time Dashboard*? Tulis evaluasi logis Anda di Laporan Praktikum (Anda bisa membahas masalah *bandwidth*, layar berkedip putih, atau potensi bentrokan jika Anda mengeklik tepat di detik ke-5).
- Kriteria Keberhasilan:
 - Nilai suhu dan kelembapan di *browser* otomatis berganti seiring kondisi lingkungan tanpa perlu mengeklik tombol *refresh* di *browser*.
 - Jawaban analisis mengenai kelemahan protokol HTTP GET statis tertuang jelas di dokumentasi tugas.



sketch_sep22a.ino

```
1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3  #include <DHT.h>
4
5  const char* ssid = "oiiu";
6  const char* password = "22222229";
7
8  ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;    // D4 / GPIO2
11 const byte relayPin = 12; // D6 / GPIO12
12
13 DHT dht(dhtPin, DHT22);
14
15 const char index_html[] PROGMEM = R"rawliteral(
16 <!DOCTYPE html>
17 <html>
18
19 <head>
20   <meta name="viewport" content="width=device-width, initial-scale=1">
21   <meta http-equiv="refresh" content="5">
22
23   <title>IoT Dashboard</title>
24
25   <body>
```

Output Serial Monitor ✕

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

```
.....
Wi-Fi Terhubung!
IP Address Server Anda: 10.97.61.112
Web Server dimulai!
```